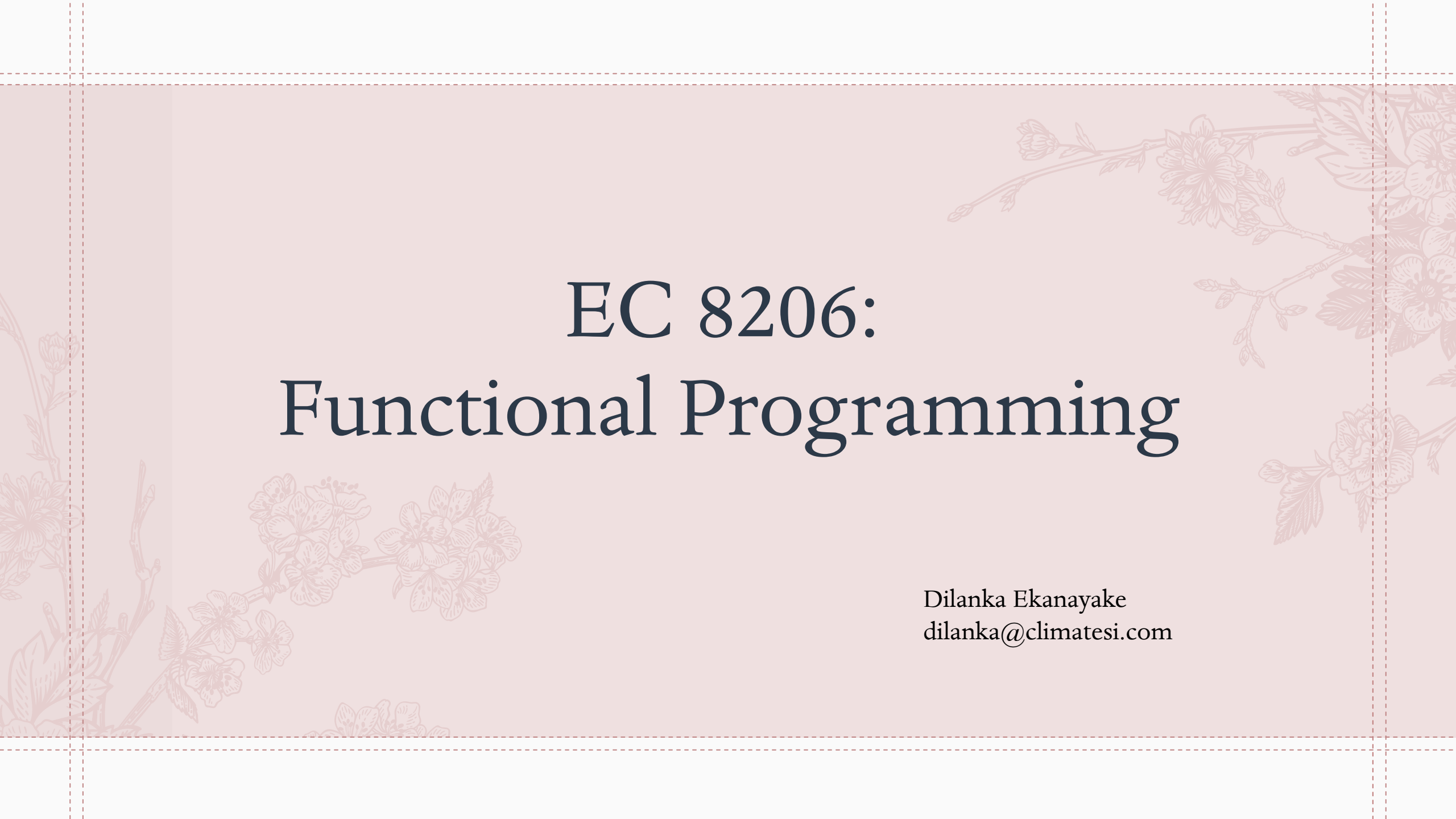




EC 8206: Functional Programming

Dilanka Ekanayake
dilanka@climatesi.com

Haskell Basics

GHC and GHCi (REPL)

Concurrency, Parallelism, Profiling

Syntax basics (expressions, functions, operators)



GHC (Glasgow Haskell Compiler)

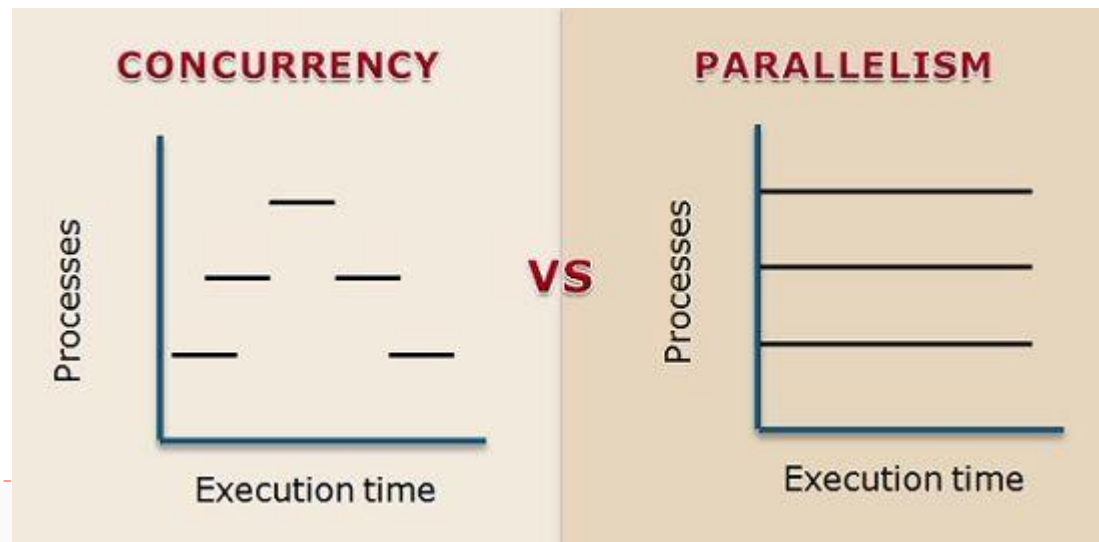
- ♦ GHC is the main Haskell compiler. It takes Haskell source code (.hs files) and compiles it into machine code, bytecode, or even JavaScript (via GHCJS).
- ♦ Support for concurrency and parallelism
- ♦ GHC has extensive optimization capabilities, including inter-module optimization.
- ♦ Profiling is supported, both by time / allocation and various kinds of heap profiling

GHCi (Glasgow Haskell Compiler Interactive)

- ♦ GHCi is GHC's interactive REPL (**Read-Eval-Print Loop**) , in which Haskell expressions can be interactively evaluated and programs can be interpreted
- ♦ GHCi can be run in a number of ways, depending on your setup and requirements:
 1. standalone: `ghci`
 2. within the stack global project: `stack repl`
 3. within a specific stack project: `cd project; stack repl`
 4. within a specific stack project, but including `GHC_PACKAGE_PATH`: `cd project; stack exec ghci`

Concurrency and Parallelism

- Concurrency involves dealing with multiple tasks simultaneously by interleaving their execution, which can happen on a single processor by quickly switching between them, while parallelism involves executing multiple tasks literally at the same time, requiring multiple processors or CPU cores to run tasks simultaneously



Profiling

- ♦ Profiling in programming is a dynamic analysis technique used to measure and analyze the runtime behavior of a program.
- ♦ It involves collecting data on various aspects of the program's execution, such as CPU time, memory usage, I/O operations, and function call frequency.
- ♦ The primary goal of profiling is to identify performance bottlenecks and resource-intensive sections of the code, allowing developers to optimize the program for better efficiency, speed, and resource utilization.
 - ♦ GHC - GHC comes with a time and space profiling system, so that you can answer questions like “why is my program so slow?”, or “why is my program using so much memory?”

Syntax and commands

<code>:! cls</code>	Clear terminal
<code>--</code>	Comment
<code>++</code>	Concatenation
<code>:quit</code>	Quit GHCi

Expressions

Numbers and arithmetic

```
Windows PowerShell
Prelude> 2 + 3
5
Prelude> 10 - 4
6
Prelude> 5 * 6
30
Prelude> 20 / 4
5.0
Prelude>
```

Booleans

```
Windows PowerShell
Prelude> True && False
False
Prelude> True || False
True
Prelude> not True
False
Prelude> 2>3
False
Prelude> |
```

Lists

```
Windows PowerShell
Prelude> [1,2,3,4]
[1,2,3,4]
Prelude> [1..8]
[1,2,3,4,5,6,7,8]
Prelude> ['a','b','c','d']
"abcd"
Prelude>
```


Functions

```
ghci> double x = x*2
ghci> square x = x*x
ghci> add a b = a+b
ghci> double 5
10
ghci> square 3
9
ghci> add 4 5
9
ghci>
```

Operators

Standard arithmetic

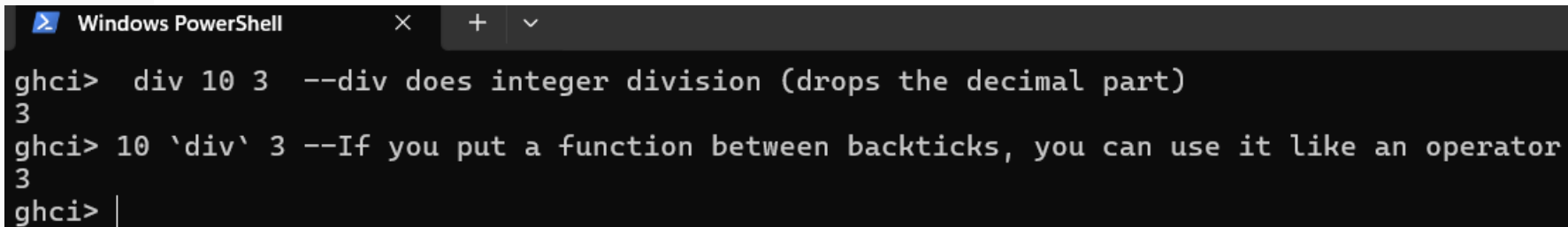
```
Windows PowerShell
ghci> 9 `div` 2    -- integer division
4
ghci> 9 `mod` 2    -- modulus
1
ghci>
```

Comparison

```
Windows PowerShell
ghci> 5 > 3
True
ghci> 5 == 5
True
ghci> 5 /= 3
True
```

Operators

Function as an operator



```
Windows PowerShell
ghci> div 10 3 --div does integer division (drops the decimal part)
3
ghci> 10 `div` 3 --If you put a function between backticks, you can use it like an operator
3
ghci> |
```

Function Composition

$(f . g) x$ means “apply g to x , then apply f to the result.”

Single composition

```
ghci> double x = x * 2
ghci> square x = x * x
ghci> (double . square) 3    -- square 3 = 9, then double 9 = 18
18
ghci> (square . double) 3    -- double 3 = 6, then square 6 = 36
36
ghci> |
```

Multiple composition

```
Windows PowerShell  X + v
ghci> double x = x * 2
ghci> square x = x * x
ghci> increment x = x + 1
ghci> (square . double . increment) 4 -- increment 4 = 5 , then double 5 = 10 and square 10 = 100
100
ghci>
```

Function Composition

Composition Using in built functions

```
Windows PowerShell
ghci> (reverse . show) 12345
"54321"
ghci>
```

Using map with composition

```
Windows PowerShell
ghci> square x = x * x
ghci> double x = x * 2
ghci> map (square . double) [1,2,3,4] -- For each element: double first, then square
[4,16,36,64]
ghci>
```

Function Composition

Composition Using in built functions

```
Windows PowerShell
ghci> (reverse . show) 12345
"54321"
ghci>
```

Using map with composition

```
Windows PowerShell
ghci> square x = x * x
ghci> double x = x * 2
ghci> map (square . double) [1,2,3,4] -- For each element: double first, then square
[4,16,36,64]
ghci>
```

Function Composition

Composition in algebra

```
Windows PowerShell
ghci> let f = (*2) . (+3)
ghci> f 7          -- (+3) 7 = 10, then (*2) 10 = 20
20
ghci> |
```

data pipeline

```
Windows PowerShell
ghci> (unlines . map reverse . lines) "dog\ncat\nfish"
"god\ntac\nhsif\n"
ghci>
```

lines → split text into lines

map reverse → reverse each line

unlines → join them back

Example 01

Write a Haskell program that:

1. Takes a list of integers from 1 to 100.
2. Finds all numbers divisible by 3 or 5.
3. Computes the sum of these numbers. Computes the sum of the squares of these numbers.
4. Returns both sums as a tuple (sum, sumOfSquares).

```
-- Define a function called sumAndSquares
-- :: IO () means this function performs Input/Output (IO) and returns "nothing useful" (unit type ())
sumAndSquares :: IO ()
sumAndSquares = do
    -- Step 1: Generate numbers from 1 to 100
    let numbers = [1..100]
    putStrLn $ "Numbers from 1 to 100: " ++ show numbers --
    -- Step 2: Filter numbers divisible by 3 or 5 using a list comprehension

    let divisible = [x | x <- numbers, x `mod` 3 == 0 || x `mod` 5 == 0] -- [ expression |
variable <- list, condition1, condition2, ... ]
    putStrLn $ "Numbers divisible by 3 or 5: " ++ show divisible
    -- Step 3: Compute the sum of divisible numbers
    let total = sum divisible
    putStrLn $ "Sum of divisible numbers: " ++ show total
    -- Step 4: Compute the sum of squares of divisible numbers
    let totalSquares = sum (map (^2) divisible)
    putStrLn $ "Sum of squares of divisible numbers: " ++ show totalSquares
    -- Step 5: Print the final result as a tuple
    putStrLn $ "Final Result as Tuple: (" ++ show total ++ ", " ++ show totalSquares ++ ")"

-- main is the entry point of a Haskell program
-- :: IO () means it performs input/output and returns unit type ()
main :: IO ()
main = sumAndSquares -- Here we are "calling" the sumAndSquares function from main
```


Example 02

Write a Haskell program that:

1. Given a principal amount p , annual rate r (as decimal), and number of years n
2. Calculate total amount after n years assuming interest is compounded annually

Example 03

Write a Haskell program that:

1. Solve quadratic equation $ax^2 + bx + c = 0$
2. Return a tuple with two roots